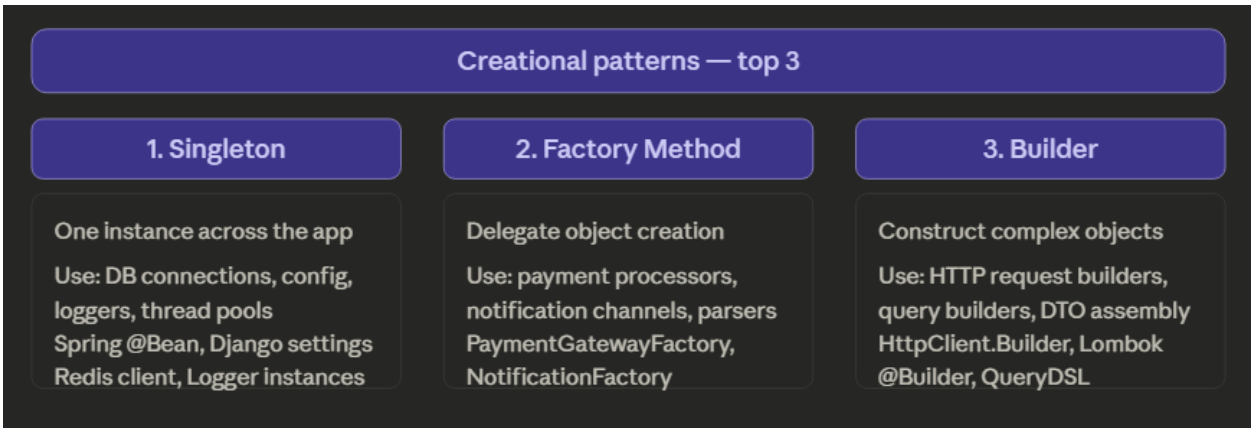
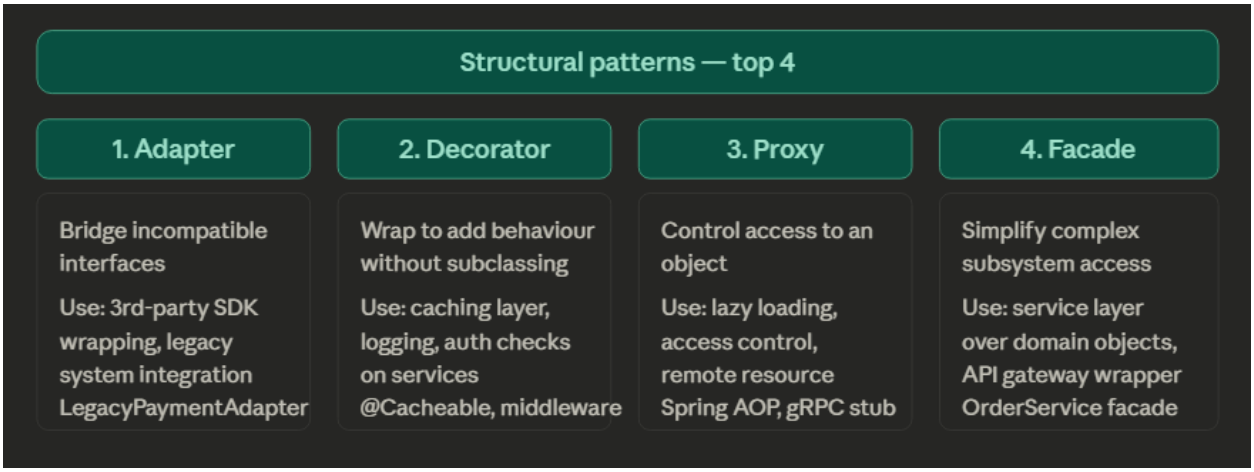
Introduction to Common Design Patterns

Types of GOF Design Patterns

These patterns are divided into three categories:

1. **Creational:** Deal with object creation (e.g., Singleton, Factory, Builder).
2. **Structural:** Deal with composition of classes/objects (e.g., Adapter, Decorator, Facade).
3. **Behavioral:** Deal with object interaction and communication (e.g., Observer, Strategy, State).





Behavioral patterns — top 4

1. Observer

Notify subscribers of state changes

Use: event systems, pub/sub, webhooks, domain events
Kafka, Spring Events

2. Strategy

Swap algorithms at runtime

Use: sorting, pricing rules, auth strategies, validation logic
Spring Security, JWT

3. Chain of Resp.

Pass request along a handler chain

Use: middleware, request pipelines, validation chains
Express.js, Servlet filters

4. Command

Encapsulate a request as an object

Use: undo/redo, job queues, CQRS write side, task scheduling
CQRS, Celery tasks

Creational Patterns

Singleton

Creational

```
DatabaseConnectionPool.getInstance()
```

WHAT IT DOES

Ensures a class has only one instance throughout the application's lifetime, and provides a global access point to that instance. Any code requesting the object always gets the same one.

USE CASES IN BACKEND DEVELOPMENT

- Database connection pool — one pool shared across the app
- Configuration manager — reads .env once, serves everywhere
- Logger — single log handler, prevents duplicate outputs
- Cache client (Redis, Memcached) — one connection, no overhead
- Service registry — one lookup map for all microservice URLs

WHEN TO USE

- A resource is expensive to create and safe to share globally
- You need a single source of truth for shared state (config, tokens)
- Coordinating access to a shared resource across modules

WHEN NOT TO USE

- The object holds mutable state that differs per user/request — use request-scoped beans instead
- In heavily concurrent systems without thread-safety guards
- When unit testing — singletons make mocking and isolation painful
- If multiple independent instances might be needed later (breaks extensibility)

Factory Method

Creational

`NotificationFactory.create('email') → EmailNotifier`

WHAT IT DOES

Defines an interface for creating an object, but lets subclasses or implementations decide which class to instantiate. The creator code never specifies the exact class — it calls a factory and gets back the right object.

USE CASES IN BACKEND DEVELOPMENT

- Payment gateway — return Stripe, PayPal or Razorpay based on region
- Notification sender — Email, SMS or Push depending on user prefs
- Auth strategy — JWT, OAuth2, SAML based on client type
- File parser — CSV, JSON or XML parser based on content type
- Report generator — PDF, Excel or HTML based on request param

WHEN TO USE

- Object creation logic should be separate from business logic
- The exact type of object won't be known until runtime
- You want to add new types without changing existing code (Open/Closed Principle)

WHEN NOT TO USE

- Only one object type will ever exist — simple instantiation is clearer
- The factory hierarchy becomes deeper than 2 levels — use a registry map instead
- Over-engineering simple CRUD services with no variation in creation

Builder Creational

```
HttpClient.newBuilder().timeout(30s).retry(3).build()
```

WHAT IT DOES

Constructs complex objects step by step. Separates the construction logic from the object's representation, allowing the same process to produce different configurations. Eliminates telescoping constructors.

USE CASES IN BACKEND DEVELOPMENT

- HTTP client config — timeouts, retries, headers, base URL
- SQL/NoSQL query building — filters, sorts, pagination, projections
- DTO / request object assembly with many optional fields
- Email message — recipients, cc, attachments, body, subject
- gRPC/Protobuf message construction with deeply nested fields

WHEN TO USE

- Object has 4+ optional/configurable fields
- Creation order matters (fluent API flow makes intent clear)
- You want immutability — build once, then the object is read-only

WHEN NOT TO USE

- Simple objects with 1–3 fields — plain constructor or static factory is cleaner
- When the object mutates frequently after creation — Builder implies immutability
- Rapid prototyping where boilerplate cost outweighs the benefit

Structural Patterns

Adapter Structural

`LegacyShippingAdapter implements ShippingPort`

WHAT IT DOES

Converts the interface of a class into another interface that clients expect. Works as a translator between two incompatible interfaces, letting them collaborate without modifying either.

USE CASES IN BACKEND DEVELOPMENT

- Wrap a third-party SDK behind your own domain interface
- Integrate a legacy payment or shipping system with a new API
- Normalize data from multiple external APIs into one internal format
- Port a service to a new cloud provider without touching domain code
- Translate between database models and API response contracts

WHEN TO USE

- Integrating external libraries without polluting domain code
- You don't own the code you're integrating (SDK, legacy system)
- Testability — wrap external calls so you can mock them easily

WHEN NOT TO USE

- You own both sides of the interface — refactor instead
- Adapting something that changes frequently — adapter becomes a maintenance burden
- Creating an adapter just to add indirection with no real incompatibility

Decorator

Structural

```
new CachingRepository(new LoggingRepository(new JpaRepository()))
```

WHAT IT DOES

Attaches additional behaviour to an object dynamically by wrapping it in a decorator object that shares the same interface. An alternative to subclassing for extending functionality.

USE CASES IN BACKEND DEVELOPMENT

- Add caching to a service method without modifying the service
- Wrap a repository with retry-on-failure logic
- Apply rate limiting or logging around HTTP client calls
- Add transaction management transparently (@Transactional)
- Compress / encrypt responses in a middleware chain

WHEN TO USE

- Adding cross-cutting concerns (logging, caching, auth) without touching core logic
- Behaviours should be composable and opt-in per instance
- Subclassing would produce an explosion of combinations

WHEN NOT TO USE

- The wrapped interface is unstable — every change cascades through decorators
- Only one behaviour is ever needed — a simple subclass is easier
- Deep decorator stacks (5+) — debugging becomes a nightmare

Facade

Structural

`OrderService.placeOrder(cart) — hides 6 internal service calls`

WHAT IT DOES

Provides a simplified, unified interface to a complex subsystem of classes. Hides internal complexity behind a clean API, so callers don't need to know how the subsystem works.

USE CASES IN BACKEND DEVELOPMENT

- OrderService coordinating inventory, payment, shipping and notification
- API gateway exposing one endpoint over multiple downstream microservices
- AuthFacade unifying JWT issuance, validation and revocation
- ReportingFacade wrapping data queries, aggregation and rendering
- BackgroundJobFacade over a queue + scheduler + worker registry

WHEN TO USE

- Exposing a simple interface over a complex domain with many moving parts
- Creating a stable public API while the internals can evolve freely
- Reducing coupling between consumer code and subsystem internals

WHEN NOT TO USE

- Consumers genuinely need fine-grained control over the subsystem
- The facade becomes a God Object with 30+ methods — split it
- Hiding complexity that consumers need to reason about (e.g., transaction boundaries)

Proxy Structural

`Spring AOP proxy intercepts @Transactional method calls`

WHAT IT DOES

Provides a surrogate or placeholder for another object to control access to it. The proxy intercepts calls and can add behaviour before/after forwarding to the real object.

USE CASES IN BACKEND DEVELOPMENT

- Lazy initialization — don't load a heavy object until first use
- Access control — verify permissions before delegating to the real service
- Remote proxy — local stub for a remote gRPC or REST service
- Caching proxy — return cached result before hitting the real DB
- AOP proxies — Spring wraps beans for @Transactional, @Async, @Retryable

WHEN TO USE

- Controlling or mediating access to a resource is the primary concern
- Adding behaviour transparently without the caller knowing
- Generating client stubs for remote services

WHEN NOT TO USE

- When Decorator already covers the need — they look similar, but Proxy controls access while Decorator adds behaviour
- When the proxy introduces latency you can't afford in hot paths
- Proxying volatile interfaces — changes break the proxy silently

Behavioral Patterns

Strategy

Behavioral

```
PricingContext.setStrategy(new TieredPricingStrategy())
```

WHAT IT DOES

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client can swap strategies at runtime without changing the code that uses them.

USE CASES IN BACKEND DEVELOPMENT

- Pricing engine — fixed, tiered, discount, subscription pricing rules
- Sorting / ranking — relevance, date, popularity, price sorts
- Authentication — JWT, OAuth2, API key, SAML as swappable strategies
- Tax calculation — different rules per country/region
- Shipping cost — courier, express, economy as strategies

WHEN TO USE

- Multiple variants of an algorithm exist and the right one is chosen at runtime
- Replacing a large if/else or switch block across business rules
- The algorithm should be independently testable and composable

WHEN NOT TO USE

- Only one algorithm exists now and you don't foresee alternatives
- The strategies are trivially simple — a plain function or map is sufficient
- Context and strategies are so tightly coupled that swapping them doesn't make sense

Observer Behavioral

```
EventBus.publish(new OrderPlacedEvent()) → 4 listeners react
```

WHAT IT DOES

Defines a one-to-many dependency so that when one object (the subject) changes state, all its dependents (observers) are notified and updated automatically. The foundation of event-driven architecture.

USE CASES IN BACKEND DEVELOPMENT

- Domain events — OrderPlaced triggers inventory, invoice, email observers
- Kafka/RabbitMQ consumers reacting to topic events
- Webhooks — notify external systems on state changes
- WebSocket push — broadcast real-time updates to connected clients
- Audit logging — any entity change triggers an audit observer

WHEN TO USE

- One event needs to trigger multiple independent reactions
- Decoupling a producer from its consumers (they shouldn't know each other)
- Building event-driven or reactive architectures

WHEN NOT TO USE

- Observer chains are long and order-dependent — use an explicit pipeline instead
- Observers have side effects that must succeed atomically — use transactions not events
- The subject and observer are always 1-to-1 — a direct call is simpler and clearer

Chain of Responsibility

Behavioral

```
AuthFilter → RateLimitFilter → ValidationFilter → Controller
```

WHAT IT DOES

Passes a request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain. Decouples sender from receivers.

USE CASES IN BACKEND DEVELOPMENT

- Middleware pipeline — auth → rate limit → validate → route → respond
- Request interceptors in Spring, Express.js, Django middleware
- Validation chain — each validator checks one rule and passes on success
- Approval workflow — junior → senior → manager approval chain
- Error handling chain — try specific handler, fall back to generic

WHEN TO USE

- Multiple handlers may handle a request and the handler isn't known ahead of time
- Handlers should be added/removed without changing the request sender
- Pipeline-style processing where each step transforms or validates the request

WHEN NOT TO USE

- Every handler in the chain must execute — that's a Pipeline, not CoR
- The chain is only ever one handler — unnecessary abstraction
- Order of handling is non-trivial and must be tightly controlled — explicit orchestration is safer

Command Behavioral

```
commandBus.dispatch(new PlaceOrderCommand(orderId, items))
```

WHAT IT DOES

Encapsulates a request as a standalone object containing all the information needed to perform the action. Enables queuing, logging, undo/redo, and deferred execution of requests.

USE CASES IN BACKEND DEVELOPMENT

- CQRS write-side — PlaceOrderCommand, CancelOrderCommand as objects
- Job queue — serialize commands to Celery, Sidekiq, BullMQ for async execution
- Undo/redo in document editors or transactional workflows
- Scheduled tasks — store a command and execute it at a future time
- Transactional outbox — persist command before publishing to message broker

WHEN TO USE

- Actions need to be queued, scheduled or executed asynchronously
- You need an audit trail or the ability to replay operations
- Decoupling the invoker of an action from its executor (CQRS)

WHEN NOT TO USE

- Simple synchronous operations with no need for queuing or replay
- Commands become bloated with business logic — they should be thin carriers, not services
- When request/response is needed immediately — async command adds latency